ResearchLens AI: PRD and Hackathon Execution Plan

Agentic AI System for Research Paper Discovery, Verification, and Scientific Reasoning (Problem 01)

---

1. Where You Are Today (Audit of the Current main.py)

Your prototype already covers a solid slice of the problem statement. This audit is what the plan is built on.

Requirement from problem statement	Status	Notes
1. Questions + PDF upload	Done	Multi-PDF upload, SHA-256 dedupe, 20 MB limit
2. Section parsing (methods, results, limitations)	Partial	Chunks carry a `section` field, but quality depends on `modules/chunking.py`; no table/caption handling visible
3. Hybrid retrieval (BM25 + dense)	Done	`HybridRetriever` with `alpha`
4. Cross-encoder reranker	Done	Only reranks against the original query, not each sub-question
5. Paragraph-level citations	Done	Numbered passages with page and section
6. Compare methods/results across papers	Missing	Only a free-text report; no structured comparison table
7. Contradiction detection	Missing	Only mentioned in the prompt, never computed
8. Claim verification	Done (basic)	`verify_answer` exists; needs per-claim entailment and citation checks
9. Follow-up with memory	Missing	Endpoints are stateless
10. Uncertainty / abstention	Partial	Only abstains when nothing is retrieved; no score threshold

Requirement from problem statement	Status	Notes
Agentic retrieval	Partial	One-shot decomposition; no iterative loop
Graph of papers/methods/findings	Missing	
Evaluation vs baselines	Missing	Biggest rubric gap (15% + part of 10%)

Production and correctness issues found in the current code

1. **Blocking endpoints.** `/ask` and `/report` are sync `def` routes that make several blocking Gemini calls. Under load they exhaust the threadpool. PDF parsing in `async def upload_pdf` also blocks the event loop.
2. **Global mutable state.** `_retriever` and `_reranker` are module globals with no lock. The global retriever is rebuilt from the whole database after every upload or delete, and `/ask` rebuilds a retriever on every request (re-tokenizing BM25 and re-embedding each time).
3. **No persistent vector index.** Dense embeddings are not stored in a vector DB, so cost grows with corpus size.
4. **Hardcoded model name** `gemini-3.5-flash-lite`. Please verify this exact string exists for your API key. Move it to config and add a fallback model so a bad name cannot break the whole pipeline.
5. **No retries, timeouts, or rate-limit handling** on Gemini calls. A single 429 returns a 502.
6. **Free-text LLM outputs parsed loosely.** Use Gemini structured output (JSON schema) for the planner, verifier, and comparison extractor.
7. **Upload edge cases.** Filename extension is the only PDF check (no magic-byte check), no page-count cap, no timeout on parsing, a failed DB write after a successful file write can race with cleanup, and stored files are never reconciled with the DB.
8. **Everything in one 700-line file.** Routing, business logic, HTML, and prompts are mixed together.
9. **Prompt injection.** Text inside uploaded PDFs is placed directly into prompts. Papers can contain instruction-like text.
10. **No auth, rate limiting, request IDs, structured logging, health checks, or tests.** `print()` is used for errors.
11. **Reranker cold start** on the first request (model load), which looks like a hang in a live demo.

---

2. Product Definition

2.1 Vision

A researcher uploads PDFs, asks a question, and receives an **evidence-backed literature report** in which every claim is cited to a paragraph, every claim is independently verified, conflicts between papers are surfaced, and the system says "I don't know" when the evidence does not support an answer.

2.2 Target users

Graduate students, researchers, and literature-review writers.

2.3 Goals

- G1: Produce a structured report: summary, method/result comparison table, consensus vs conflicts, research gaps, limitations, uncertainty.
- G2: Zero tolerance for fabricated citations. Every citation must resolve to a real stored passage.
- G3: Measurable quality with baselines (BM25 only, dense only, no reranker, plain LLM).
- G4: Runs reliably from one `docker compose up`.

2.4 Non-goals (for the 48h scope)

- Multi-tenant SaaS billing, OAuth, or enterprise SSO.
- Fine-tuning models.
- OCR of scanned PDFs (detect and reject with a clear message; stretch goal only).

2.5 Success metrics

Metric	Target	Dataset
Retrieval Recall@10 / nDCG@10	Hybrid + rerank beats BM25-only and dense-only	SciFact corpus, QASPER
Answer faithfulness	At least 0.85 (claims supported by cited passages)	Your own test set, RAGAS-style
Citation precision / recall	At least 0.90 / 0.80	Your own test set
Claim verification F1	Report vs a baseline	SciFact

Metric	Target	Dataset
Abstention accuracy	At least 80% on unanswerable questions	QASPER unanswerable + synthetic
P95 latency, `/ask`	Under 25 s (3 papers); report under 60 s	Load test

---

3. Technology Decisions

The suggested stack in the problem is a menu, not a requirement. Choices below optimise for **a 48-hour build that still looks and behaves like production**.

Layer	Choice	Why	Rejected
API	**FastAPI** (keep)	Already built; async, typed, auto docs	Flask
Orchestration	**LangGraph**	Planner → retrieve → rerank → synthesize → verify → loop is a graph with conditional edges (iterative search, retry, abstain). Matches the reference architecture and is a visible "agentic" feature for judges	LangChain chains (too linear), hand-rolled loops (hard to debug and trace)
LLM	**Gemini via `google-genai`**	Required. Use **structured output** (Pydantic `response_schema`) everywhere, a strong model for synthesis/verification and a cheap Flash-Lite for planning	Open-source LLMs (infra cost and time)
PDF parsing	**PyMuPDF** (+ `pymupdf4llm` for tables)	Fast, runs locally, gives page numbers, bounding boxes, and table extraction. Fallback to Gemini document understanding for hard pages	GROBID (excellent section parsing but needs a Java service; make it a stretch goal)

Layer	Choice	Why	Rejected
Sparse retrieval	**rank_bm25** (in-memory, per-collection, cached)	Matches the suggested stack; fine at hackathon scale	Elasticsearch (overkill)
Dense embeddings	**`BAAI/bge-small-en-v1.5`** locally, or `allenai/specter2` for scientific text. Optional: Gemini embedding API	Free, no per-call cost, strong on retrieval benchmarks. Run an A/B on SciFact and keep the winner	Gemini embeddings as default (cost and latency per chunk)
Vector store	**Qdrant** (Docker, persistent)	Persistent, metadata filtering by `document_id` (needed for "search only selected papers"), production-grade	ChromaDB (fine, but weaker filtering and persistence story at scale)
Reranker	**`cross-encoder/ms-marco-MiniLM-L-6-v2`** (keep) or `BAAI/bge-reranker-base`	Compare both in eval	LLM reranking (slow and costly)
Metadata DB	**SQLite now, SQLAlchemy ORM** so Postgres is a config change	Zero setup, migration path ready	Raw SQL strings
Background jobs	**FastAPI BackgroundTasks + job table** (stretch: Redis/RQ)	Ingestion is slow; return a `job_id` and poll	Doing it inline in the request
Frontend	**Streamlit** for the 48h demo, calling the FastAPI API	Fastest path to a chat UI with citations, tables, and graphs. Keep API UI-agnostic so React can replace it	React (more polish, more hours than you have)
Graph visual	**NetworkX + pyvis** (rendered in Streamlit)	Paper–method–finding graph with little code	Neo4j
Eval	**Custom harness + `ragas`-style prompts + `ir_measures`/own nDCG**	Reproducible scripts with saved JSON results	Manual spot checks

Layer	Choice	Why	Rejected
Quality	`pytest`, `ruff`, `mypy` (light), `pydantic-settings`	Standard	
Observability	`structlog` + request IDs, per-stage timing, token counting	Needed for debugging and for the demo	`print()`
Packaging	**Docker + docker compose** (api, qdrant, ui)	One-command setup, which the submission checklist requires	

---

4. Target Architecture

```
PDF upload
  → validate (magic bytes, size, pages)
  → parse (PyMuPDF) → section detector → table/caption-aware chunker
  → embed → Qdrant (vectors + metadata) + BM25 index cache + SQLite (docs, chunks)

Question
  → LangGraph agent:
      [Planner] decompose question into sub-questions
         ↓
      [Retriever] hybrid (BM25 + dense), filtered to selected documents, per sub-question
         ↓
      [Reranker] cross-encoder against each sub-question, merge, dedupe
         ↓
      [Sufficiency check] enough evidence? no → reformulate and loop (max 2 iterations)
                                                  still no → ABSTAIN
         ↓
      [Extractor] structured findings per paper (method, dataset, metric, result, limitation)
         ↓
      [Comparator] comparison table + contradiction/consensus detection
         ↓
      [Synthesizer] cited answer using only passage IDs
         ↓
      [Verifier] per-claim entailment against cited passages; citation validity check
```

```
          ↓
      [Report builder] summary, table, consensus vs conflicts, gaps,
limitations, uncertainty
  → UI (chat, report, graph, evidence viewer)
```

Proposed repository structure

```
researchlens/
├── app/
|   ├── main.py                  # app factory, lifespan (warm-up models),
middleware
|   ├── config.py                # pydantic-settings (models, limits,
thresholds)
|   ├── api/
|   |   ├── deps.py              # shared dependencies (db session, services)
|   |   └── routes/
|   |       ├── documents.py     # upload, list, get, download, delete
|   |       ├── jobs.py          # ingestion status
|   |       ├── search.py
|   |       ├── ask.py           # /ask, /conversations/{id}/messages
|   |       ├── report.py
|   |       └── health.py        # /health, /ready
|   ├── core/                    # logging, errors, retries, rate limiting
|   ├── schemas/                 # all Pydantic request/response models
|   ├── db/                      # models, session, repositories
|   ├── ingestion/               # validator, parser, sections, chunker,
indexer
|   ├── retrieval/               # bm25, dense, hybrid, reranker,
index_manager
|   ├── agents/                  # LangGraph graph + nodes
|   |   ├── graph.py
|   |   └── nodes/               # planner, retriever, sufficiency,
extractor,
|   |                            # comparator, synthesizer, verifier,
reporter
|   ├── llm/                     # gemini client wrapper, retries, structured
output, prompts/
|   └── services/                # conversation memory, citation service,
graph builder
├── ui/streamlit_app.py
├── eval/                        # datasets, runners, baselines,
results/*.json, report.md
```

```
├── tests/                        # unit, integration, fixtures (small PDFs)
├── docker-compose.yml, Dockerfile, .env.example, Makefile, README.md
└── docs/                         # architecture diagram, limitations, failure
cases
```

---

5. Functional Requirements

FR-1: Ingestion

- Accept 1 to N PDFs; validate magic bytes (`%PDF-`), size, page limit (e.g. 100 pages), and non-empty extractable text.
- Return `202` with a `job_id` ; per-file status: `queued → parsing → chunking → embedding → ready | failed` .
- Detect sections (Abstract, Introduction, Related Work, Methodology, Experiments, Results, Discussion, Limitations, Conclusion, References) by font size/weight and heading regex. Assign `section` to every chunk and drop or flag the references section.
- **Chunking rules:** keep a table with its caption as one chunk; keep figure captions with adjacent text; target 200 to 400 tokens with small overlap; never split mid-sentence; store `page_start` , `page_end` , `chunk_index` , `chunk_type` (text / table / caption).
- Extract metadata when possible (title, authors, year) for display and for the comparison table.

FR-2: Retrieval

- Hybrid score fusion (weighted or Reciprocal Rank Fusion; evaluate both, default to the winner).
- Filter by selected `document_ids` inside the vector query, not after.
- Per-sub-question reranking, then merge with dedupe and per-paper diversity cap so one paper cannot dominate.

FR-3: Agentic reasoning

- Planner returns 2 to 5 sub-questions via structured output; fallback to the original query.
- Sufficiency check after reranking: if the best rerank score is under the threshold or the sub-question has no evidence, reformulate and retry (max 2 loops), then abstain for that part.

FR-4: Comparison and contradiction detection

- Extractor produces a structured record per paper: `{method, dataset, metric, value, baseline, limitation, passage_ids}`.
- Comparator builds a Markdown/JSON table across papers; values are always tied to passage IDs.
- Contradiction detector: for pairs of findings on the same dataset/metric/claim, run an NLI-style Gemini judgment (`supports | contradicts | unrelated`) and label output as **Consensus**, **Conflict**, or **Insufficient evidence**.

FR-5: Verification and citation integrity

- Split the answer into atomic claims; for each claim verify against its cited passages (`supported | partially_supported | unsupported`).
- **Citation validity check (deterministic):** every `[n]` in the answer must map to a real passage in this request's evidence. Strip or flag invalid ones.
- Unsupported claims are either removed in an automatic repair pass or clearly marked in the UI.

FR-6: Uncertainty and abstention

- Per-answer confidence: combine rerank scores, verification support ratio, and evidence coverage (papers and sub-questions covered) into `high | medium | low`.
- Abstain with an explanation of what is missing when confidence is under threshold.

FR-7: Conversation memory

- `conversation_id` with stored turns. Follow-ups are rewritten into standalone questions using recent history ("What about its limitations?" becomes a full query) before planning.
- Memory scoped per conversation with a turn cap and summary for long chats.

FR-8: Report generation

Structured JSON plus rendered Markdown, with these sections: literature summary; method/result comparison table; consensus vs conflicts; research gaps; limitations of the evidence; uncertainty analysis; references with page-level citations. Export as Markdown and PDF.

FR-9: Paper graph (advanced)

Nodes: papers, methods, datasets, findings. Edges: `uses`, `evaluated_on`, `reports`, `contradicts`, `supports`. Built from the extractor output; shown in the UI.

---

6. API Contract (v1)

Method	Path	Purpose
POST	`/v1/documents`	Upload PDFs, returns `job_id`(s)
GET	`/v1/jobs/{job_id}`	Ingestion status and progress
GET	`/v1/documents`	List with status, pages, chunks
GET / DELETE	`/v1/documents/{id}`	Details / delete (DB, vectors, file)
GET	`/v1/documents/{id}/download`	Original PDF
POST	`/v1/search`	Retrieval-only (for eval and debugging)
POST	`/v1/conversations`	Start conversation
POST	`/v1/conversations/{id}/messages`	Ask (with memory), returns answer, citations, verification, confidence
POST	`/v1/reports`	Generate structured report (async job for large inputs)
GET	`/v1/graph?document_ids=`	Paper/method/finding graph
GET	`/health`, `/ready`	Liveness and readiness (models and Qdrant loaded)

Standard error body: `{ "error": { "code": "...", "message": "...", "request_id": "..." } }`.

---

7. Edge Case Matrix

Area	Edge case	Expected behaviour
Upload	Non-PDF with `.pdf` extension	Reject via magic-byte check

Area	Edge case	Expected behaviour
Upload	Empty file, over 20 MB, over page limit	Per-file failure with a clear reason; other files continue
Upload	Encrypted / password-protected PDF	Fail with "PDF is encrypted"
Upload	Scanned PDF with no text layer	Fail with "No extractable text (scanned?)"
Upload	Corrupt or truncated PDF	Parser exception caught; file removed; reported
Upload	Duplicate content, different filename	Detected by SHA-256; return the existing document
Upload	Same filename, different content	Allowed; stored by `document_id`
Upload	Path traversal in filename	Filename sanitised; stored as UUID
Upload	Two identical uploads at the same time	Unique constraint on hash; second becomes a duplicate result
Upload	Crash between file write and DB write	Compensating cleanup plus a startup reconciliation job
Parsing	Two-column layout	Column-aware block ordering
Parsing	Table split from its caption	Table + caption chunked together
Parsing	References section dominating retrieval	Tagged and excluded by default
Parsing	Ligatures, hyphenation, odd Unicode	Normalised
Retrieval	Zero documents selected / document missing	`422` / `404` with clear message
Retrieval	Query in another language or only stopwords	Graceful empty result, then abstain
Retrieval	Very long query	Length cap, then truncate or summarise
Retrieval	One paper dominates results	Per-paper diversity cap
Retrieval	Document deleted while a query is running	Index version check; retry or skip deleted IDs
LLM	429 / 5xx / timeout	Exponential backoff with jitter, then fallback model, then partial result with a warning

Area	Edge case	Expected behaviour
LLM	Invalid or non-JSON output	Structured output plus one repair retry, then deterministic fallback
LLM	Safety block or empty response	Detected; return a clear message instead of 502
LLM	Context too large	Token budgeting; trim lowest-scored passages
LLM	Hallucinated `[n]` citation	Deterministic citation validator removes or flags it
Security	Prompt injection inside PDF text	Wrap evidence in delimiters, instruct the model to treat it as data, scan for instruction-like patterns, never allow tools to be called from evidence
Security	API key exposure	Only from env; never logged; `.env` in `.gitignore`
Security	Abuse / flooding	Per-IP rate limit; optional API key header
Memory	Follow-up with ambiguous pronoun and no history	Ask a clarifying question
Memory	Follow-up after selected documents changed	Conversation is bound to its document set; warn on change
Answering	Evidence contradicts itself	Present both sides labelled **Conflict**; do not average
Answering	Question unanswerable from the papers	Abstain and state what is missing
Answering	Question about the paper's authors' intent, not in text	Mark as not established
Ops	Cold start of reranker / embedder	Warm-up in lifespan; `/ready` only true when loaded
Ops	Disk full / Qdrant down	`/ready` fails; clear 503

---

8. Evaluation Plan

This is the most under-built area and worth about 25% of the rubric (model performance plus data engineering). Plan it early.

Datasets

- **SciFact**: retrieval (Recall@k, nDCG@10) and claim verification F1.
- **QASPER**: paper QA with answerable and unanswerable questions (abstention accuracy, answer F1).
- **Custom set (30 to 50 questions)** over 5 to 8 papers you choose, including multi-paper comparison, contradiction, and unanswerable questions. This protects you from "only testing on easy questions."

Baselines (report all in one table)

1. BM25 only
2. Dense only
3. Hybrid without reranker
4. Hybrid + reranker (yours)
5. Plain Gemini with no retrieval (to show hallucination)
6. Full agentic pipeline with the verifier

Metrics: Recall@5/10, nDCG@10, faithfulness (fraction of claims supported), citation precision and recall (cited passage truly supports the claim), claim-verification F1, abstention accuracy, latency and token cost per query.

Reproducibility: fixed seeds, versioned dataset splits, `make eval` runs everything and writes `eval/results/*.json` plus `eval/report.md`. Never tune thresholds on the test split (leakage prevention).

---

9. Responsible AI and Explainability

- Show the exact source passage, page, and section for every citation; click to open the PDF page.
- Label every claim with a verification badge and show the overall confidence.
- State limitations in the UI: the system only knows the uploaded papers, can misparse tables, and is not a replacement for reading the paper.
- Do not present LLM-inferred statements as findings; unsupported content is removed or flagged.

---

10. Hackathon Execution Plan (48 Hours)

Because the prototype already works, hours are re-allocated from "build basics" to "restructure, add agentic and verification depth, and evaluate." Suggested roles for a team of 3 to 4: **A** backend/ingestion, **B** retrieval/eval, **C** agent/LLM, **D** UI/demo (merge roles if smaller).

Phase 0 (0 to 4h): Foundation and refactor

- Create the repo structure from section 4; split `main.py` into routers, services, schemas, and config.
- Add `pydantic-settings`, `.env.example`, structured logging, request IDs, error handler, `/health`.
- Dockerfile and docker-compose (api + qdrant).
- Gemini client wrapper: retries, timeouts, fallback model, structured output helper.
- **Exit criteria:** existing endpoints work unchanged from the new structure; `docker compose up` runs.

Phase 1 (4 to 12h): Ingestion and index (parallel: A and B)

- Hardened validator, async ingestion jobs, job status endpoint.
- Section detector and table/caption-aware chunker; reference exclusion.
- Qdrant dense index with document filtering; cached BM25 per collection; incremental index updates (no rebuild-all).
- Model warm-up in lifespan.
- **Exit criteria:** 10 sample papers ingest correctly; sections look right on a manual check of 3 papers; tables stay with captions.

Phase 2 (12 to 22h): Retrieval and agent core (B and C)

- RRF vs weighted fusion experiment; per-sub-question reranking; diversity cap.
- LangGraph graph: planner, retriever, reranker, sufficiency check with iterative loop and abstention.
- Structured findings extractor.
- **Exit criteria:** graph runs end to end on a multi-paper question and logs each node's output and timing.

Phase 3 (22 to 32h): Generation, comparison, citations (C and A)

- Synthesizer using passage IDs only; deterministic citation validator.
- Comparison table and contradiction/consensus detector.

- Conversation memory with follow-up rewriting.
- Structured report builder (all sections in FR-8) and Markdown/PDF export.
- **Exit criteria:** report contains every required section; 0 invalid citations on the test set.

Phase 4 (32 to 40h): Verification and uncertainty (C and B)

- Per-claim verifier with repair pass; confidence scoring; abstention thresholds (tuned on a dev split only).
- Prompt-injection hardening; rate limiting.
- Paper/method/finding graph (stretch if time allows).
- **Exit criteria:** verifier flags planted wrong claims; unanswerable questions abstain.

Phase 5 (40 to 48h): Evaluation, polish, demo (everyone)

- Run the full baseline comparison; write `eval/report.md`.
- Streamlit UI: upload and job progress, chat, report view, citation viewer, verification badges, graph tab.
- README with setup, architecture diagram, dataset and preprocessing docs, limitations and failure cases (with real examples you observed).
- Record the demo; rehearse a 5-minute presentation: problem, approach, results, future work.
- **Exit criteria:** fresh-clone setup works in under 10 minutes; checklist below fully ticked.

Cut line (if time runs short, drop in this order)

1. Paper graph visual. 2. PDF export. 3. Gemini-fallback parsing. 4. RRF vs weighted experiment. 5. Async job queue (keep a simple background task).

Never cut: evaluation with baselines, citation validator, verification, abstention, README with failure cases.

---

11. Rubric Alignment

Rubric criterion	Weight	How this plan earns it
Problem understanding	10%	Audit, user definition, requirement traceability (sections 1 and 5)
Technical depth	20%	Hybrid retrieval, cross-encoder, LangGraph iterative agent, per-claim NLI verification, contradiction detection

Rubric criterion	Weight	How this plan earns it
Data engineering	10%	Section/table-aware chunking, dedupe, versioned splits, no threshold tuning on test data, reproducible `make eval`
Model performance	15%	Six-way baseline table on SciFact, QASPER, and a custom hard set
System architecture	15%	Modular repo, async ingestion, persistent vector DB, Docker, health checks
Working prototype	10%	Streamlit UI plus recorded demo
Explainability and responsible AI	10%	Verifiable citations, badges, confidence, abstention, documented limitations
Innovation and scalability	5%	Agentic loop, paper graph, Postgres/Qdrant migration path
Presentation	5%	Scripted 5-minute demo

---

12. Submission Checklist

- [ ] GitHub repo with setup instructions (`.env.example` , one-command run)
- [ ] Working prototype or recorded demo
- [ ] Architecture documentation with diagram
- [ ] Dataset sources and preprocessing methodology
- [ ] Evaluation results with baseline comparisons
- [ ] Limitations and failure cases (with real examples)
- [ ] One technically challenging feature demonstrated (recommended: the agentic loop with claim verification and contradiction detection)
- [ ] Short presentation (problem, approach, results, future work)

---

13. Risks and Mitigations

Risk	Likelihood	Mitigation
Gemini rate limits or quota during demo	High	Caching of planner/extractor outputs, retries, fallback model, pre-ingested demo papers, recorded backup demo

Risk	Likelihood	Mitigation
Bad section detection on some PDFs	Medium	Heading heuristics plus fallback to a generic section label; show limitations honestly
Over-scoping (graph, GROBID, React)	High	Cut line in section 10; Streamlit only
Eval takes too long to run	Medium	Subsample SciFact for dev, run full eval once at hour 40
Verifier is too strict or too lenient	Medium	Calibrate on a small labelled dev set; report precision/recall of the verifier itself
Model name or API changes	Low	Models in config; startup check lists available models

---

14. Immediate Next Steps (First 2 Hours)

1. Confirm the Gemini model strings available to your key and put them in config (primary, cheap, fallback).
2. Share `modules/chunking.py`, `retrieval.py`, `database.py`, and `verification.py` so the refactor and section-detection work can be reviewed against the real code.
3. Decide team roles and create the repo skeleton.
4. Pick 8 to 10 papers for the custom evaluation set now, since that set drives the demo and the metrics.